

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron by transforming the weighted sum of inputs into an output value. It enables a neural network to perform classification and decision-making tasks. In this experiment, the Step Activation Function was used for the perceptron implementation, while the Sigmoid Activation Function is presented for comparison.

3.1 Step Activation Function

The Step Activation Function produces a binary output based on whether the input is above or below a threshold.

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad (1)$$

Characteristics:

- Produces binary outputs (0 or 1).
- Used in classical perceptrons.
- Non-differentiable at $x = 0$.
- Gradient is zero almost everywhere.
- Cannot be trained using backpropagation.

4. Dataset Description

Dataset	Banknote Authentication Dataset
Source	UCI Repository
Instances	1372
Features	4
Classes	2
Missing Values	None

Features

- Variance
- Skewness
- Curtosis
- Entropy

5. Experimental Procedure

1. Dataset loading : We load(insert) the data using which the single-layer perceptron must perform the classification.
2. Exploratory Data Analysis : This is to understand the trends in the data and relationships between features so the classification has a basis to work on and not classify randomly.

3. Data Pre-Processing : Raw data might contain some dirty values which disrupt the learning. To avoid any disruption, data cleaning and scaling is done in this step.
4. Train-test split : Training and testing on the same data is not suitable as it would lead to memorization. To regularize the results, the data is split and a small unseen portion is used for testing to mimic real-world data.
5. Perceptron implementation : The weight initialization, bias initialization, Activation Function, Forward Pass, and the Perceptron Learning Rule.
6. Model training : Training on the training data by repeatedly predicting classes, checking if the prediction is correct, and altering the weight and bias parameters by repeatedly performing forward passes and parameter updates using the Perceptron Learning Rule for a fixed number of times or till the error in predictions is zero or less than some threshold.
7. Performance evaluation : The learning of the perceptron is measured by testing it with the unseen testing data. Metrics like Accuracy, Precision, Recall and F1 score are used.

6. Exploratory Data Analysis

6.1 First Five Samples

	0	1	2	3	4
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Figure 1: First Five Samples

The columns are variance, skewness, kurtosis, entropy and the class of each row of data. The data is accurate with 4-5 decimals.

6.2 Dataset Statistics

	0	1	2	3	4
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Figure 2: Descriptive Statistics

6.3 Histogram

Histograms of Banknote Authentication Features

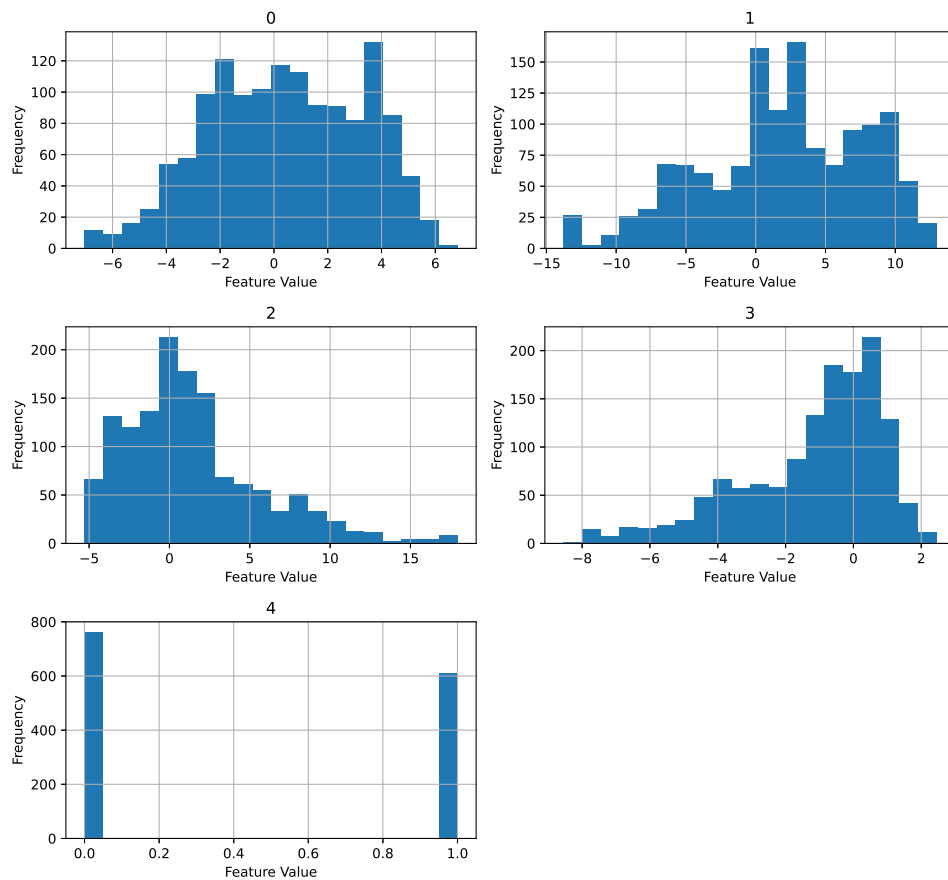


Figure 3: Feature Histogram

The histograms show the distribution of each feature in the dataset. The distribution is non-uniform, suggesting that feature normalization could prove to be helpful.

6.4 Correlation Heatmap

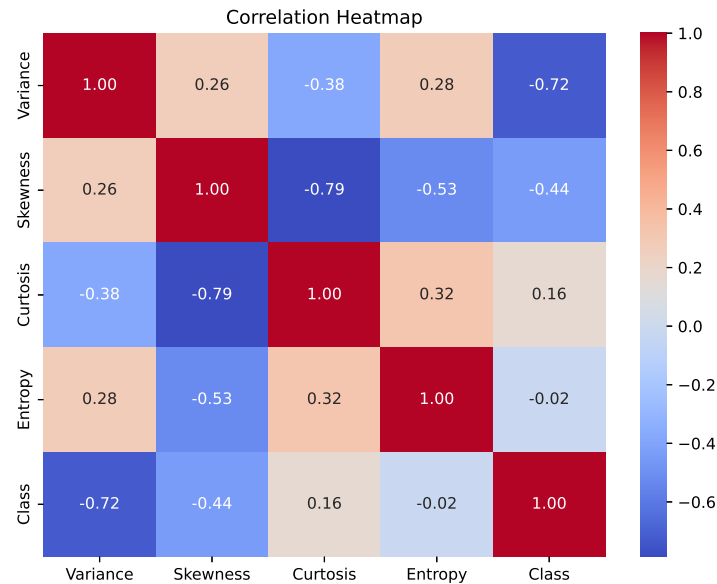


Figure 4: Correlation Heatmap

The heatmap shows the relationship between features. Two feature pairs show good correlation, one feature pair shows nearly no correlation, while the rest of the feature pairs show moderate correlation.

6.5 Scatter Plot

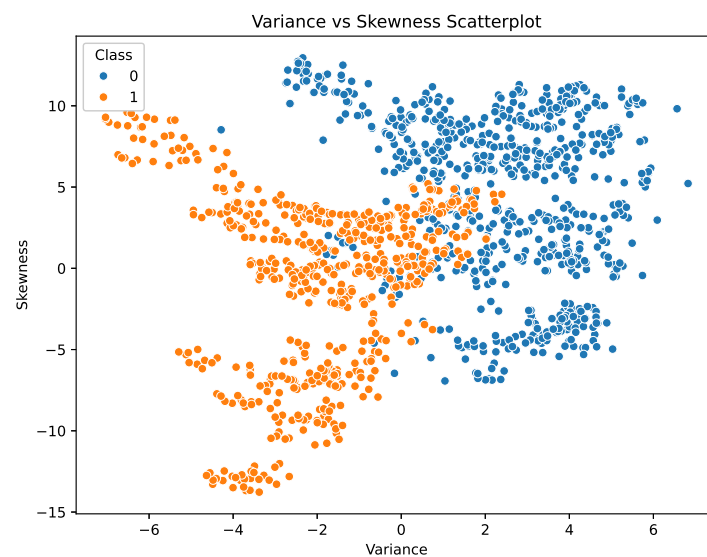


Figure 5: Scatterplot

The plot shows a considerable hyperplane can be drawn using just these two features.

6.6 Boxplots

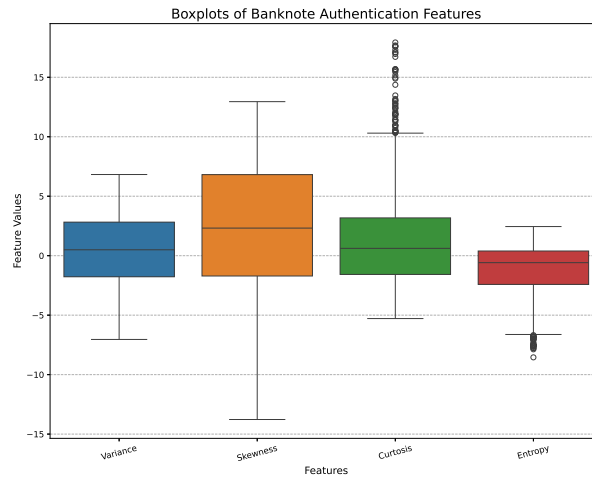


Figure 6: Boxplot

The box plots summarize the distribution of the features numerically by showing the median, inter-quartile range and outliers.

7. Data Preprocessing

- Feature Scaling : Transforming the features numerically to have the same range of values to make the contributions of each feature equal.
- Train-Test Split : Splitting the dataset into two subsets: the training set and the testing set. The training set is used to learn the parameters and the testing set is used to assess the performance.
- Normalization : A type of scaling technique in which the range is between 0 and 1 or the mean is 0 and variance is 1.

8. Perceptron Implementation

- Weight initialization : Weights refer to the importance that each input feature is given. They are usually initialized with zeros or random values. They are updated throughout the training course to reduce errors.
- Bias initialization : It is an additional parameter that improves the flexibility of the perceptron. It is also initialized with zeros or random values and updated like the weights are.
- Forward propagation : The process of computing a prediction with a given input. The input values of the features are multiplied by the weights, added with the bias, and given to the activation function to produce the predicted class.
- Step activation : Converts the weighted sum of the inputs into a binary output. If the sum is greater than 0, the output is 1, otherwise the output will be 0.
- Weight update : This is the learning process of the perceptron. The weights and bias are adjusted according to the Perceptron Learning Rule to reduce errors in predictions of classes.

9. Source Code

The complete source code for this experiment, including dataset preprocessing, EDA, Perceptron implementation, model training, and evaluation, is available in the following GitHub repository:

https://github.com/vai-bhav5335/Deep_Learning_Lab/tree/main

The repository contains all Python scripts, generated plots, and supporting files used in this experiment.

10. Training Results

10.1 Epoch-wise Learning

Epoch	Errors	W_1	W_2	W_3	W_4	Bias
1	47	-0.072537	-0.064863	-0.064460	0.000199	-0.03
2	26	-0.082170	-0.093802	-0.051494	-0.005182	-0.03
3	34	-0.094676	-0.091400	-0.089055	-0.009565	-0.05
5	27	-0.130964	-0.121796	-0.091128	-0.008771	-0.05
10	26	-0.131687	-0.155810	-0.126006	0.011535	-0.06
20	17	-0.156418	-0.174663	-0.155186	-0.025960	-0.09
30	18	-0.175740	-0.187043	-0.178879	-0.013906	-0.10
40	21	-0.174245	-0.215330	-0.197946	-0.014856	-0.10
50	13	-0.196595	-0.219351	-0.210659	-0.018478	-0.10

Table 1: Epoch-wise Learning

10.2 Training Error vs Epoch

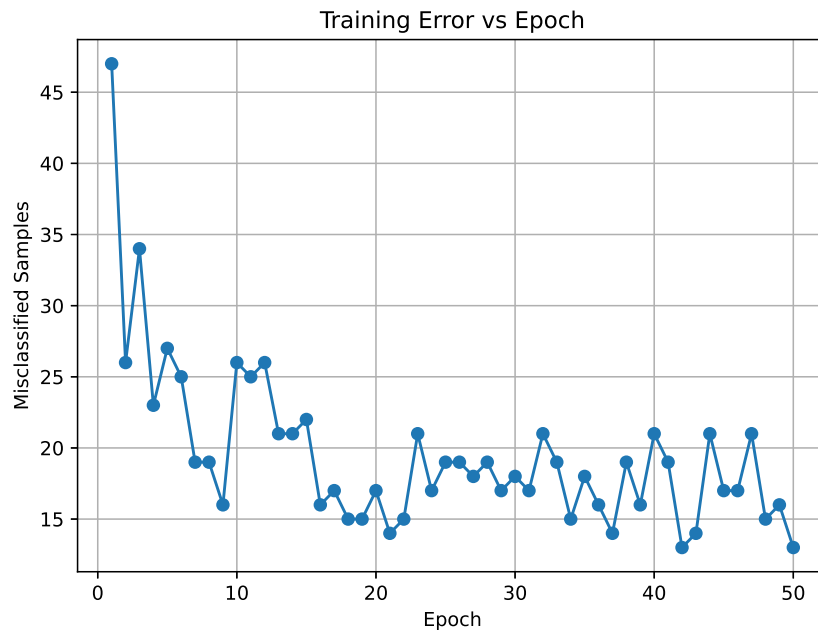


Figure 7: Training Error vs Epoch

The training error reduces drastically in the first few epochs, and gradually, but not consistently for the rest of the epochs.

10.3 Weight Evolution

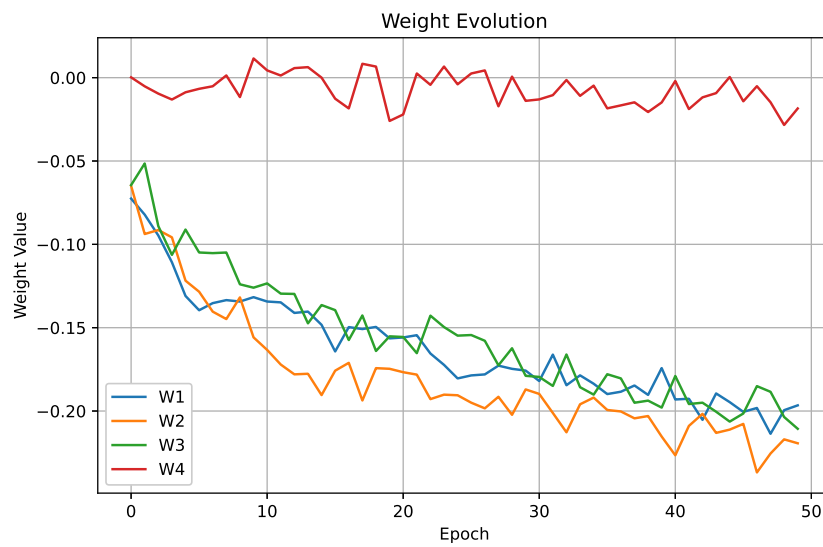


Figure 8: Weight Evolution

There is not much importance given to input feature 4 which is Entropy, but the other input features have nearly equal importance.

10.4 Bias Evolution

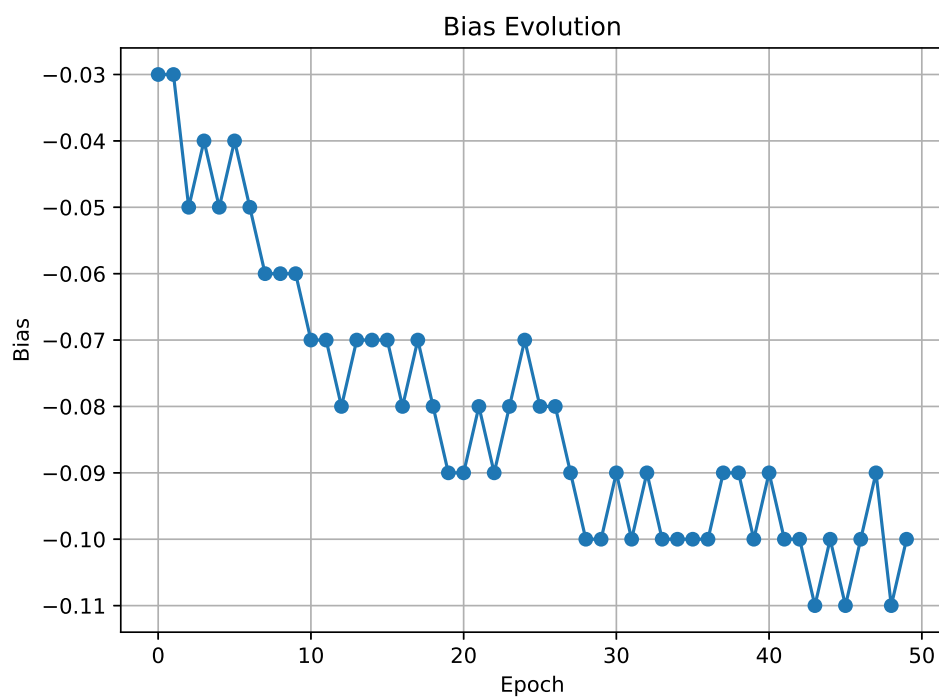


Figure 9: Bias Evolution

The bias gradually decreased in value with a few kinks in the middle.

11. Model Evaluation

11.1 Performance Metrics

Accuracy	0.978182
Precision	0.984
Recall	0.968503
F1 Score	0.976190

Table 2: Performance Metrics

11.2 Confusion Matrix

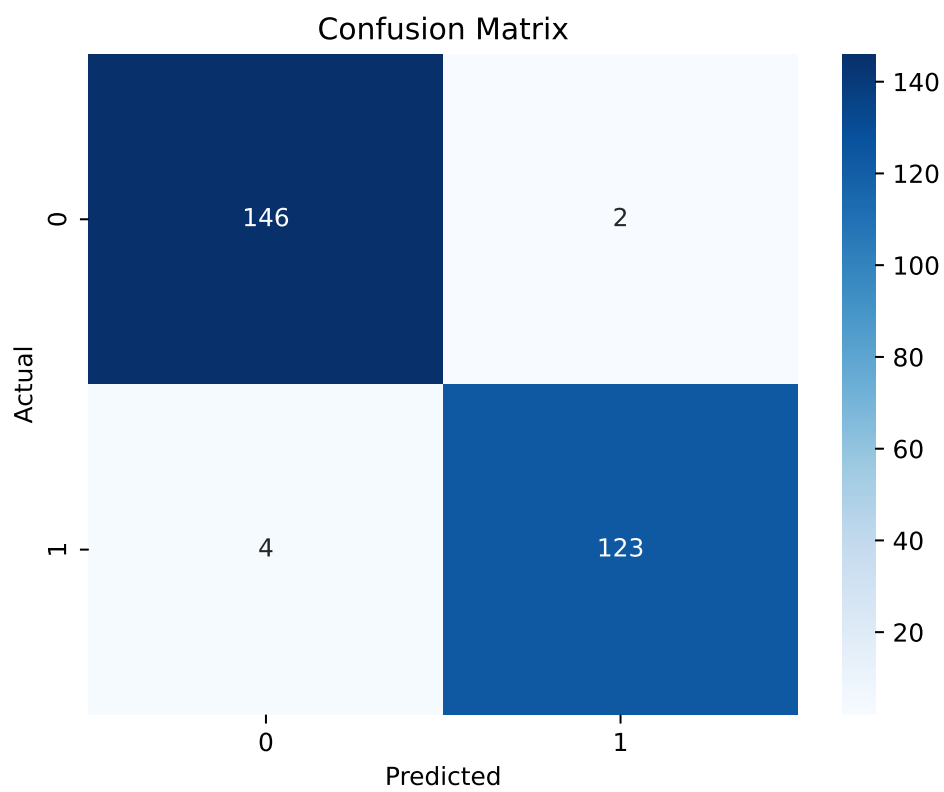


Figure 10: Confusion Matrix

Observation.

11.3 Learning Rate Comparison

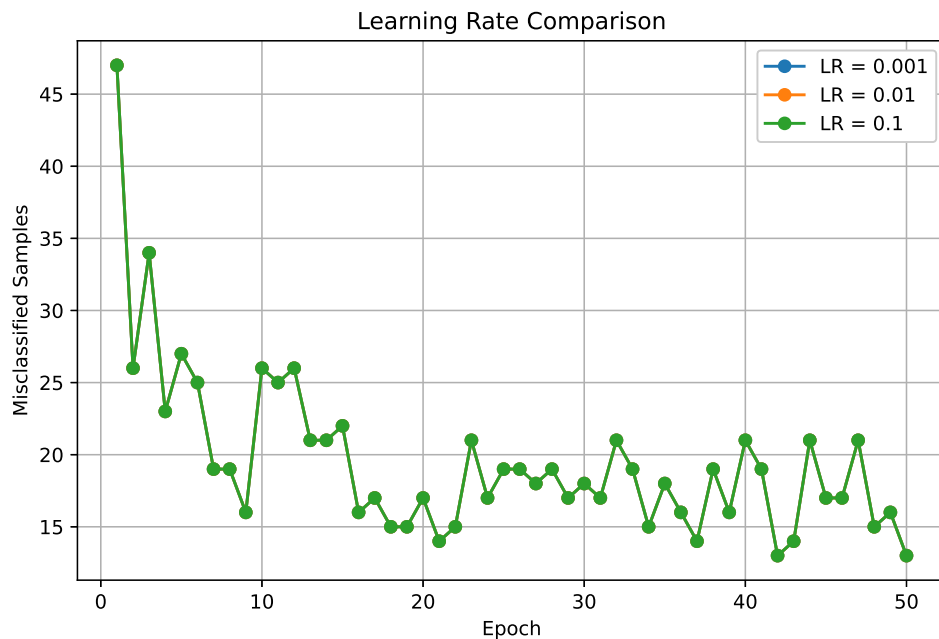


Figure 11: Learning Rate Comparison

This plot shows that the curves obtained for different learning rates are identical, showing that the selected learning rates had the same influence on the convergence and all 3 were within a range for which the learning is stable.

12. Discussion

- **Model convergence :** The process by which the perceptron model reduces its errors and reaches a set of weights and bias which result in making the best predictions. While training, the number of errors decreases over successive epochs, showing that the model was learning the decision boundary effectively. Since this Banknote Authentication dataset is approximately linearly separable, the perceptron is able to converge successfully. Once proper convergence is achieved, further training produces no change in the model parameters.
- **Training behavior :** The perceptron updates its weights whenever it classifies a sample incorrectly. As training progresses, the number of updates in the weights decreases because the model grows better at classifying the data. This results in improved prediction accuracy. The observed behavior demonstrates how the perceptron learns a linear decision boundary using the training data.
- **Effect of normalization :** Improved stability and speed of convergence of the perceptron. Contributes to better training efficiency and reliable model performance.
- **Effect of learning rate :** A small learning rate results in slow convergence because of minor adjustments. A large learning rate causes oscillations which also slows training. Here, the acceptable range for a good learning rate is large because of the linear separability of the data and the normalization of the features.
- **Why XOR cannot be solved :** If the XOR values are mapped on a graph, it shows that no straight line can separate the classes cleanly, meaning the XOR problem is not linearly separable. The single layer perceptron can only work with problems that are linearly separable, so it can never achieve zero errors while solving the XOR problem.

13. Conclusion

This experiment was to implement a Single Layer Perceptron from scratch for a binary classification problem using the Banknote Authentication dataset. The experiment included loading the dataset, performing exploratory data analysis, preprocessing the data, splitting the data into a training set and a testing set, implementing the Perceptron Learning Algorithm, training the model and evaluating its performance using standard classification evaluation metrics.

The exploratory data analysis provided insights into the distribution of features, the relationships between them, and the outliers present. Feature normalization kept all input variables on the same scale resulting in faster and stable convergence. The Perceptron Learning Algorithm successfully updated the weights and bias iteratively to minimize errors, demonstrating how a linear classifier learns a decision boundary from labeled data.

The model achieved satisfactory performance on the dataset after training, indicating that the dataset is mostly linearly separable. By monitoring the training error, weight evolution and bias evolution across epochs, the learning process of the perceptron was clearly observed and analyzed.

Overall, this experiment provided a good understanding of the working of the Single Layer Perceptron, including data preprocessing, weight updates, bias updates, convergence and performance evaluation. It established a strong foundation for studying and understanding more advanced neural network architectures used in modern Deep Learning.

14. References

1. F. Rosenblatt, The Perceptron.
2. Goodfellow, Bengio and Courville, Deep Learning.
3. Bishop, Pattern Recognition and Machine Learning.
4. Haykin, Neural Networks and Learning Machines.
5. UCI Machine Learning Repository.
6. Scikit-learn Documentation.